Computer Systems

Spring 2026

Tony Stone

Contents

1 Preface **3**
1.1 Mission Statement . . . 3
1.2 Acknowledgments . . . 3
1.3 On the Content . . . 3

2 Binary Numbers **4**
2.1 Numbers in General . . . 4
2.1.1 Decimal . . . 4
2.1.2 Binary . . . 4
2.1.3 Hexidecimal . . . 4
2.2 Converting number systems . . . 5
2.2.1 Binary to Decimal . . . 5
2.2.2 Hexidecimal (to Binary) to Decimal . . . 5
2.2.3 Decimal to Binary . . . 5
2.3 Binary Arithmetic . . . 6

3 Boolean Logic **8**
3.1 Basic Operations . . . 8
3.1.1 AND . . . 8
3.1.2 OR . . . 9
3.1.3 NOT . . . 9
3.2 Warming up to Boolean Algebra . . . 9

4 Boolean Algebra **11**
4.1 Laws . . . 11
4.1.1 Double Negation . . . 11
4.1.2 Complement . . . 11
4.1.3 Associative Laws . . . 11
4.1.4 Commutative Laws . . . 12
4.1.5 Distribuve Laws . . . 12
4.1.6 Identity Laws . . . 12
4.1.7 Annihilator Laws . . . 12
4.1.8 Idempotence . . . 12
4.1.9 De Morgan's Laws . . . 12

4.2 Simplifying Boolean Expressions 13
4.2.1 Some Problems 13

5 Minterms **14**
5.1 minterms 14

6 K-Maps **15**
6.1 Anatomy of the K-Map 15

1 Preface

1.1 Mission Statement

With this work, I set out to develop a comprehensive set of lecture notes, on the topic of Computer Systems. While developing these notes, I am keeping in mind both the lecturer, and the learner. My goal is that this will amount to an educational resource that acts as a guide throughout fundamental topics, in a manner which is accessible and painless for all parties. This work should provide confidence and direction to its reader, whether they are at the front of a classroom, or at their dining-room table.

1.2 Acknowledgments

I would like to thank the Center for Academic Success and Accessibilty Services at Southern Connecticut State University, for the employment and opportunity to develop these materials. I would like to extend this thank you for all of the administrative support I have recieved to my boss Kathleen De Oliveira, and coordinaters Tanisha Guadalupe De Jésus, Calleigh Cotter, and Mia Grella. Of course, I would like to extend this thank you to Professor Lisa Lancor, for allowing me to attend her CSC-207 lectures and lab.

1.3 On the Content

Computer Systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language by Dr. Ata Elahi—a professor at SCSU—was the textbook used for the CSC-207 Course in the spring session. This textbook, as well as the points emphasized during lecture provide the foundations for the material that appears in these lecture notes.

2 Binary Numbers

2.1 Numbers in General

In general, any number can be represented as:

$$(a_n ... a_2 a_1 a_0 . a_{-1} a_{-2} ... a_{-n})_r$$

Where r denotes the base, and a_i must be less than r. When n is *positive*, that corresponds to the *integer* portion of the number. when n is negative, that corresponds to the *fractional* portion of the number. (I am avoiding the word 'decimal' as it is used differently in the bottom, and I do not want to confuse the two). This expands into

$$a_n \times r^n + ... + a_1 \times r^1 + a_0 \times r^0 + a_{-1} \times r^{-1} + ... + a_{-n} \times r^{-n}$$

2.1.1 Decimal

Numbers are represented in different **bases**. You are likely most familiear with *base ten* also known as *decimal*. For example consider the number 524, and note:

$$524 = 500 + 20 + 4 = 5 \times 100 + 2 \times 10 + 4 \times 1$$

The key is to notice:

$$5 \times 100 + 2 \times 10 + 4 \times 1 = 5 \times 10^2 + 2 \times 10^1 + 4 \times 10^0$$

2.1.2 Binary

. A **binary** number is the number of the form:

$$(b_n ... b_2 b_1 b_0 . b_{-1} b_{-2} ... b_{-n})_2$$

Where each digit b_i is either a 0 or a 1. and the base is 2. For example, the number $(101)_2$. For binary, i'll be referring to each digit as a **bit**.

2.1.3 Hexidecimal

. An **hexidecimal** number is a number of the form:

$$(h_n ... h_2 h_1 h_0 . h_{-1} h_{-2} ... h_{-n})_{16}$$

Where h_i is one of the following digits:

$$0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$$

For example, the number $(A2)_{16}$. The following table shows decimal, binary, and hexidecimal numbers all together. Note how each hexidecimal digit corresponds to a 4-bit binary number—we'll leverage this for doign base conversions.

Decimal	Binary	Hexidecimal
0	0000	0x0
1	0001	0x1
2	0010	0x2
3	0011	0x3
4	0100	0x4
5	0101	0x5
6	0110	0x6
7	0111	0x7
8	1000	0x8
9	1001	0x9
10	1010	0xA
11	1011	0xB
12	1100	0xC
13	1101	0xD
14	1110	0xE
15	1111	0xF

2.2 Converting number systems

A motivation for knowing about number systems, is that we can convert between them, which helps how we encode and represent certain kinds of informaiton.

2.2.1 Binary to Decimal

To convert from decimal to binary, we will take a given bit b_1, and multiply it by 2^i. We will sum all of these terms, and arrive at a number in base 10, which we will represent as $()_{10}$. Consider the above number $(101)_2$

$$(101)_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 1 \times 4 + 0 \times 2 + 1 \times 1 = 4 + 0 + 1 = 5$$

2.2.2 Hexidecimal (to Binary) to Decimal

Let's look back on our number from the above example: $(A2)_{16}$. Like we had said, the hexidecimal digits correspond to a 4-bit binary number:

$$A2_{16} = (10100010)_2$$

Which we just learned how to convert!

$$(10100010)_2 = 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$= 128 + 32 + 2 = 162$$

2.2.3 Decimal to Binary

So what if we want to represent a decimal number as a binary number? We'll set up an **EQR table**, which stands for *Equation, Quotient, Remainder*, as those things are exactly what we'll be keeping track of!

We begin by setting up a table with three columns, one to do integer division (we get integers with a remainder), another to get the quotient, and one more to record the remainder. Each step, we will divide our decimal number by 2 and record its quotinet and remainder. In the next step, we'll take that quotient and divide it by 2, and so on. Lets say we wanted to convert $(23)_{10}$ to its binary equivalent:

Equation	**Quotient**	**Remainder**
$23 \div 2$	11	1
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

Now what? Take the remainder column, and then 'knock it over,' something like:

$$\begin{matrix} b_0 \\ b_1 \\ \dots \\ b_n \end{matrix} \quad \rightarrow \quad b_n ... b_1 b_0$$

For our above example:

$$\begin{matrix} 1 \\ 1 \\ 1 \\ 0 \\ 1 \end{matrix} \quad \rightarrow \quad (10111)_2$$

And there we go! A word of warning: it is very easy to overlook that final row, and forget that $1 \div 2$ is 0 with a remainder of 1, or to forget to do it entirely! (definitely not speaking from experience).

I encourage you to always double check: just do a quick conversion back to decimal with the binary result you got, the decimal mumber should equal what you started with.

$$\begin{aligned} (10111)_2 &= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1 \\ &= 16 + 0 + 4 + 2 + 1 \\ &= (23)_{10} \end{aligned}$$

All good!

2.3 Binary Arithmetic

I think it will be best to get a conceptual point of binary addition out of the way; here goes:

$$0 + 0 = 0$$

(obviously)

$$1 + 0 = 0 + 1 = 1$$

(of course)

$$1 + 1 = 10$$

...

If that looks a little wacky, thats okay! Let's unpack this a bit. These numbers are binary, so they're of the form $(b_n ... b_0)_2$. if we converte these calculations to Decimal, we can see that this all tracks.

Binary	**Decimal**
$(0)_2 + (0)_2 = (0)_2 \rightarrow$	$0 \times 2^0 + 0 \times 2^0 = (0)_{10} + (0)_{10} = (0)_{10}$
$(0)_2 + (1)_2 = (1)_2 \rightarrow$	$0 \times 2^0 + 1 \times 2^0 = (0)_{10} + (1)_{10} = (1)_{10}$
$(1)_2 + (1)_2 = (10)_2 \rightarrow$	$1 \times 2^0 + 1 \times 2^0 = (1)_{10} + (1)_{10} = (2)_{10}$

So for binary, $1 + 1 = 10$. We see that in decimal, this equates to 2. Which makes sense! Converting $(10)_2$ to decimal:

$$(10)_2 = 1 \times 2^1 + 0 \times 2^0 = 2 + 0 = (2)_{10}$$

This example will become trivial in no time, especially when we extend this to addition on larger strings of binary numbers!

3 Boolean Logic

Boolean logic is a way of designing functions with variables that are either one of two values. In formal logic, these would be "True" or "False". For us, these values will be 1 or 0.

But like any function, we define them in terms of variables. I will use A, B, ... Z for these variables. The following subsections will be descriptions of the operations we will use to define boolean functions, and the notation that I will be following.

- **Tip**: For the basic operators AND, OR, and NOT, you can think them out by simply using them in a scentence, something like "C is 1 because A AND B are both 1."

3.1 Basic Operations

The Basic operations will be outlined in this section, these are the fundamental operators that will be used to define the many gates and circuits we will be looking at.

We will begin understanding these operations through **truth tables**. It is a table, whose column headers correspond to variables (or expressions) and the remainder of the columns show all the combinations of possible values for those variables, like so:

A	B	$operation$
0	0	1/0
0	1	1/0
1	0	1/0
1	1	1/0

Get very used to drawing this table, you will over, and over, and over, and over again.

3.1.1 AND

AND is the operation that evaluates to 1, when both inputs are 1. For an actual expression, I will use the '.' symbol to denote AND. Let's see that in a truth table:

A	B	$A.B$
0	0	0
0	1	0
1	0	0
1	1	1

3.1.2 OR

OR is the operation that evaluates to 1 as long as either input is equal to 1. The operator symbol I will use for OR will be the '+' sign. This may be a little tricky since we normally use this as shorthand for 'and' in english, but here, this means OR! Let's see the Truth table:

A	B	$A+B$
0	0	0
0	1	1
1	0	1
1	1	1

Now we see that there are additional cases when $A+B$ equals 1. It may be tricky to think about how the last one could work, but remember our definition: *as long as one input equals* 1, and both is certaintly at least one!

3.1.3 NOT

Perhaps the easiest to intuit is the NOT operation. This operation switches the value of whatever variable or expression it belongs to. I'll use an apostrophy to represent NOT. So $NOTA$ will look like: A'.

A	A'
0	1
1	0

3.2 Warming up to Boolean Algebra

When evaluating a boolean expression, you will be given an expression, and then a series of values. For now, treat parenthesis just like pempdas: evaluate whatever is inside first, and go from there. Note:

$$(A.B)' \neq A'.B'$$

But we will discuss more on this later. Let's take a look at a few examples! (remember, if you ever feel stuck, refer to a truth table–you can never draw enough!).

Consider the following expressions, suppose $A = 0$ and $B = 1$

1. $A + B$
$$A + B = 0 + 1 = 1$$
2. $A.B + B$
$$A.B + B = 0.1 + 1 = 0 + 1$$
3. $A' + B'$
$$A' + B' = 0' + 1' = 1 + 0 = 1$$

But we won't use expressions, we will be evaluating **boolean functions**. We'll still be evaluating *an* expression with given values, but we will define an expression as F, where the variables are passed as parameters. Let's do some more solving:

Consider the following expressions, suppose $A = 0$ and $B = 1$

1. $F(A, B) = (A.B)' + A'$

$$F(0, 1) = (0.1)' + 0' = 0' + 0' = 1 + 1 = 1$$

2. $F(A, B) = (A'.B').(A'.B)$

$$F(0, 1) = (0'.1').(0'.1) = (1.0).(1.1) = (0).(1) = 0.1 = 0$$

4 Boolean Algebra

With a labratory component, you will often be shown that boolean functions can be translated into logical circuitry, and vice-versa. While this is a fascinating insight, we don't want to stop here.

Just like normal expressions, there are often simpler ways to express the same exact thing. In algebra class, you would not leave $\frac{24}{3}$ as an answer, or use it for further solving. You would simplify it to 8. While it is the same *value*, it is easier to interpret and to use.

Think of this at the high level; if two functions have identical truth tables, are they logically different? No, as a matter of fact, we call this a **logical equivalence**. the **laws** are ways that allow us to change an expression to preserve logical equivalence.

4.1 Laws

In this section, we'll explore the kind of algerbraic moves that we can make with boolean expressions. For anyone who is taking, or has taken, a discrete mathematics/logic course will immediately pick up on these!

As with any algebraic move, we are looking for an *alternative* way to represent *the same thing*. With this new representation, we are hoping to either gain more helpful information, or to make it more useful to us. This is particularly clear with simplificaton as we stated previously, we can make things simpler, which is likely more efficient when mapped to actual circuitry.

A great resource is the Wikipedia page on boolean algebra. I'll be naming and referring to these laws as they are referred to in this page, but I will stick with our agreed notation.

4.1.1 Double Negation

$$(A')' = A$$

The negation of a negation of a variable is equivalent to the variable.

4.1.2 Complement

$$A.A' = 0$$

The AND operation on a variable and it's negation will always be 0.

$$A + A'$$

The OR operation on a variable and it's negation will always be 1.

4.1.3 Associative Laws

$$(A + B) + C = A + (B + C)$$
$$(A.B).C = A.(B.C)$$

The grouping of three variables in the OR or the AND operaiton.

4.1.4 Commutative Laws

$$A + B = B + A$$
$$A.B = B.A$$

4.1.5 Distribuve Laws

$$A + (B.C) = (A + B).(A + C)$$
$$A.(B + C) = A.B + A.C$$

This one is definitely a conceptual leap from the other ones. Just like regular distribution of the negative sign in algebra class, we do a similar distribution, just wiht other operations. I assure you you have already seen this, we'll do it in a little more explicitly to get the idea.

$$-(3 + 4) = -(7) = -7$$

$$-(3 + 4) = (-1)(3 + 4) = (-1 \times 3) + (-1 \times 4) = (-3) + (-4) = -7$$

4.1.6 Identity Laws

$$A + 0 = A$$
$$A.1 = A$$

These are called identity, because the expression preserves the identity of whatver variable is in this operation.

4.1.7 Annihilator Laws

$$A + 1 = 1$$
$$A.0 = 0$$

Contrarily, these expressions anihilate the identity of the variable.

4.1.8 Idempotence

$$A + A = A$$
$$A.A = A$$

These ones seem wrong, like it shouldn't even be possible. I assure you it is, remember to refer to truth tables if you find these particulary un-intuitive!

4.1.9 De Morgan's Laws

$$(A + B)' = A'.B'$$
$$(A.B)' = A' + B'$$

Speaking of un-intuitive, you *cannot* merely distribute negation. It must be done in this specific way.

4.2 Simplifying Boolean Expressions

Now that we have build this toolbox of algebraic moves, let's apply it to some practice problems! Try to connect these equations back to a logical circuit!

An important note is that you can think of these laws as working two ways. Often, we will use the distributive law, not to necessarily distribute, but to collect a term that is already distributed. You will see in the following examples.

Another note, when we are solving equations, we should specify every single step, and the specific law that we are using at every step. But isn't this a pain? Invariably, but it makes for better understanding, and prvoides an explicit line of thinking. It's wins all around.

4.2.1 Some Problems

1. $F(A, B) = A.(A' + B)$

$$\begin{aligned}
&A.(A' + B) = A.A' + A.B && \textit{Distributive law}\\
&= 0 + A.B && \textit{Complement law}\\
&= A.B && \textit{Identity law}
\end{aligned}$$

2. $F(A, B) = B.(A + B)' + A'$

$$\begin{aligned}
&B.(A + B)' + A' = B.(A'.B') + A' && \textit{De Morgan's law}\\
&= B.A'.B' + A' && \textit{Associative law}\\
&= B.B'.A' + A' && \textit{Commutative law}\\
&= 0.A' + A' && \textit{Complement law}\\
&= 0 + A' && \textit{Annihilator law}\\
&= A' && \textit{Identity law}
\end{aligned}$$

3. $F(A, B, C) = A.B.C + C.A.B'$

$$\begin{aligned}
&A.B.C + C.A.B' = A.B.C + A.B'C. && \textit{Commutative law}\\
&= A.(B.C + B'.C) && \textit{Distributive law}\\
&= A.(C.B + C.B') && \textit{Commutative law}\\
&= A.(C.(B + B')) && \textit{Distributive law}\\
&= A.C.(B + B') && \textit{Associative law}\\
&= A.C.(1) && \textit{Complement law}\\
&= A.C.1 && \textit{Associative law}\\
&= A.C && \textit{Identity law}
\end{aligned}$$

4. $F(A, B) = (A.B)'.(A + B)'$

$$\begin{aligned}
&(A.B)'.(A + B')' = A.B + (A + B') && \textit{De Morgan's law}\\
&= A.B + A + B' && \textit{Associative law}\\
&= A.B + A.1 + B' && \textit{Identity law}\\
&= A.(B + 1) + B' && \textit{Distributive law}\\
&= A.(1) + B' && \textit{Annihilator law}\\
&= A.1 + B' && \textit{Associative law}\\
&= A + B' && \textit{Identity law}
\end{aligned}$$

5 Minterms

for a given boolean function, we can represent it as a sum of **minterms**, and as a product of **Maxterms**. (I like to always write minterms with a lower case, since we'll use variables m_i to denote them, and an uppercase for Maxterms, as we'll deonte them with M_i.)

5.1 minterms

Consider a function $F(A, B)$ (we don't need to know what it is just yet). F, has 4 max terms. In fact, for any boolean function, the number of maxterms is $2^{number\ of\ variables}$. Here, we see it is $2^2 = 4$ minterms.

A **minterm** is an AND opperation on the inputs. For minterms, a 0 is represented by the complement of a variable. For a 1, it is just the variable.

What are the possible combinations for this function? Well, this is what our truth table tells us! For a general function F, the output is going to be a minterm, labeled m_i. We fill the following table:

A	B	$minterm$	$F(A, B)$
0	0	$A'.B'$	m_0
0	1	$A'.B$	m_1
1	0	$A.B'$	m_2
1	1	$A.B$	m_3

Now, Say you don't Know what F is, but you know the following truth table:

A	B	$F(A, B)$
0	0	1
0	1	0
1	0	1
1	1	0

We can express F as a sum of minterms, as described in the ealrier table. A 1 signifies that that minterm is a part of the sum, and a 0 means that it is not.

For this function:

$$F(A, B) = m_0 + m_2$$

and therefore,

$$F(A, B) = A'.B' + A.B'$$

Let's apply some boolean algebra to simplify F!

$$\begin{aligned}
&A'.B' + A.B' = B'.A' + B'.A && \textit{Commutative law} \\
&= B'.(A' + A) && \textit{Distributive law} \\
&= B'.(1) && \textit{Complement law} \\
&= B'.1 && \textit{Associative law} \\
&= B' && \textit{Identity law}
\end{aligned}$$

referring to the earlier table, we do in fact see the F is simplly the complement of B!

6 K-Maps

Karnaugh Maps (K-Maps) provide a way to simplify a boolean expression, without all of the algebra.

6.1 Anatomy of the K-Map

Each cell of a K-map represents a minterm. Each row and column will correspond with a 0 or 1 for each variable. Recall that 0 represents a negation, and that 1 represents that variable.

The number of variables n, corresponds to 2^n cells. So, a boolean expression that has 2 variables will have $2^2 = 4$ cells. An expression with three variables will have $2^3 = 8$ cells, and so on. If this is a little confusing, that's okay! We'll walk through the two variable example first, and build from there.

Consider two boolean variables, A and B. We'll organize a table in the following fashion:

	$0(B')$	$1(B)$	B
$0(A')$	m_0 $(A'.B')$	m_1 $(A'.B)$	
$1(A)$	m_2 $(A.B')$	m_3 $(A.B)$	
A			

In order to make some meaning out of this, we will fill the table out according to some function F, and the value of it's minterms. We will then group this function based on cells that contain a 1 and are adgacent. This can be:

- Cells that are directly on top of each other, or directly side by side.
- Single cells

However, we may **not** take:

- Diagonal cells.

Tempting as it may be, these will simply have to be two single cell groups.

Now, say we have a boolean function $F(A, B)$, and we know that the minterm $m_0 = 1$, and $m_2 = 1$. Well, we can update our K-map to represent this:

	$0(B')$	$1(B)$	B
$0(A')$	$m_0 = 1$	$m_1 = 0$	
$1(A)$	$m_2 = 1$	$m_3 = 0$	
A			

Here, we would be able to group all of the B' column, since the cells contain 1s, and are adjacent. The idea to notice, is that this has shown use that no matter what value A has (A' or A), we take B'. So, the expression F simplifies to:

$$F = B'$$

That's really the main takeaway; by using a K-Map, we're able to cleany visualise what variables are not changing

References

[1] Ata Elahi. *Computer Systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language*. Springer Cham, 2022.